KG College of Arts and Science

KGCAS co-Kreate your Genius

KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

C PROJECT REPORT ON

"Unit convertor suite"

SUBMITTED TO

The Technical Department

KG College of Arts and Science

SUBMITTED BY

Student Name	Roll No.	Course & Batch
Bagavathi	25bcs106	cs-2025
Bavya sree	25bcs107	cs-2025
Dhanushya	25bcs108	cs-2025
Dharshini	25bcs109	cs-2025
Dharshini priya	25bcs110	cs-2025

Academic Year: **2025 – 2026**

Semester: **I**

DECLARATION

We hereby declare that this mini project report entitled "**Unit Converter Suite**" has been carried out by us under the guidance of our faculty. This project is a part of our course requirements and is submitted in partial fulfillment of the award of our degree/diploma.

We also declare that this report is the result of our own work and efforts. The information and data used in this project are genuine to the best of our knowledge, and this report has not been submitted elsewhere for any other academic purpose

Student Name	Roll No.	Course & Batch	Signature
Bagavathi.	25bcs106	cs-2025	V. Bhagy
Bavya sree	25bcs107	cs-2025	P. Bavy
Dhanushya	25bcs108	cs-2025	Dhanushya. B
Dharshini	25bcs109	cs-2025	A. Dharshini
Dharshini priya	25bcs110	cs-2025	A. Dhrpr.

Place: COIMBATORE

Date: 16.03.2026

CERTIFICATE

This is to certify that the mini project report entitled "**UnitConverter Suite** " has been successfully completed and submitted by the students of the **Department of Computer Science**, in partial fulfillment of the requirements for the award of the degree/diploma during the academic **year 2025– 2026.**

This project has been carried out under the supervision and guidance of the Project Guide, **Department of Computer Science.**

Project Team:

S. No.	Name	Roll Number
1.	Bagavathi	25bcs106
2.	Bavya sree	25bcs107
3.	Dhanushya. B	25bcs108
4.	Dharshini	25bcs109
5.	Dharshini priya. A	25bcs110

ABSTRACT

The **Unit Converter Suite** is a simple and efficient program developed in the C programming language to perform various unit conversions quickly and accurately. The main objective of this project is to create a tool that allows users to convert values between **different measurement units such as length, temperature,**
weight, time, and volume.

The program provides a user-friendly menu-driven interface that makes it **easy for users to select** the desired conversion type and input values. It applies standard **mathematical formulas** to ensure precise results. This project helps beginners understand fundamental C programming concepts such as functions, **conditional**
statements, loops, and switch-case structures.

Overall, the **UnitConverterSuite** serves as a practical and educational application that demonstrates how basic programming techniques can be used to **solvereal-worldproblemsefficiently.**

Table of content

S. No.	Topics	Page. No.
1	Introduction	1
1.1	Objective	2
1.2	Overview	3
1.3	Features	4
2	Program Analysis	5
2.1	Problem Definition	6
2.2	Project Overview	7
2.3	Scopes and Limitations	8
3	System Design	9
3.1	Modules	10
4	Methodology	11
4.1	Problem Analysis	12
4.2	System Design	13
4.3	Program Development	14
4.4	Testing and Validation	15
4.5	Result and Documentation	16
4.6	Algorithm	17

4.7	Flowchart	18
5	Source Code	19
6	Input and Output Screens	20
7	Discussion and Conclusion	21
7.1	Discussion	22
7.2	Conclusion	23
8	References	24

1. **INTRODUCTION**

A **Unit Converter Suite** is a menu-driven C program that converts values between different measurement units across categories such as **length, mass, temperature, volume, and time**. This project demonstrates modular design, user interaction, and basic numerical operations in C.

1.1 Objective

•Implement a reliable and user-friendly unit conversion program in C.

•Cover multiple unit categories (length, mass, temperature, volume, time).

•Demonstrate modular programming, input validation, and documentation.

1.2 Overview

•The program provides a main menu listing conversion categories.

•Each category opens a sub menu of conversions.

•Users can perform repeated conversions until they exit.

•The program prints results with appropriate precision.

1.3 Features

•Menu-driven interface

•Multiple categories: Length, Temperature, Mass, Volume, Time

•Input validation for menu choices

•Clear input/output screens and sample test cases

•Modular functions for each conversion category

1. PROGRAM ANALYSIS

The program reads user selection, requests **theinputvalueanddesiredconversiontype,performsarithmetic conversion, andprintstheresult.** Conversions are implemented using constant factor multipliers or simple formulas (e.g., C↔F).

2.1 Problem Definition

Create a single **Cprogram** that allows converting units across commonly-used measurement categories through an **easy-to-use** menu interface.

2.2 Project Overview

Aspect	Description
Language Used	C
Compiler	GCC / Dev-C++ / Turbo C++
Concepts Used	Functions, Loops, Switch Case, Conditional Statements, Input/Output Operations, Arithmetic Operations, Modular Programming
I/O Type	Console-based
Optional	User-friendly Interface, Accurate Conversions, Expandable Categories

2.3 Scopes and Limitations

Scope:

- Support main unit categories and common units within them.

Limitations:

- Floating point rounding errors are possible due to double precision.
- The program runs in console; no GUI. Does not persist history
- between runs.

1. SYSTEM DESIGN

3.1 Modules

A modular approach where each conversion category is a separate function. The **main()** provides the primary menu and routes control.

1. **main_menu()** — displays top-level menu

2. **length_menu()** — length conversions

3. **temperature_menu()** — temp conversions

4. **mass_menu()** — mass conversions

5. **volume_menu()** — volume conversion

6. **time_menu()** — time conversions

7. **get_double()** — helper for safe double input

Module Interaction Overview:

Separate modules for each conversion type.

Main module handles user input and directs flow.

Conversion modules return results to main module.

Simple, reusable, and easy to expand.

1. METHODOLOGY

1.Design **menus and conversion maps**

2.Implement **conversion functions with clear formulas**

3.Validate **inputs and loop** until user exits

4.Test with **known values and edge cases**

4.1. Problem Analysis

In **day-to-day life** and scientific work, values are expressed in various measurement units such as meters, kilometers, miles, Celsius, Fahrenheit, kilograms, and pounds. However, these units are not always compatible, which makes calculations and comparisons difficult. Manually converting between units can lead to confusion and errors, especially when multiple conversions or large data sets are involved.

To solve this problem, a Unit Converter Suite is proposed — a C-based program that allows users to easily and accurately convert values from one unit to another. The system is designed to handle various categories such as length, weight, temperature, and time.

4.2. System Design

The Unit Converter Suite is designed using a modular approach with three main parts.

1. **Input Module** – takes user's choice and value.

2. **Processing Module** – performs unit conversion using base-unit formulas.

3. **Output Module** – displays the converted result clearly.

Flow:

User → Main Menu → Conversion Logic → Result Display

The system ensures accurate conversion, easy navigation, and simple error handling for invalid inputs.

4.3. Program Development

The Unit Converter Suite Program was developed using the C language. The program uses a **menu-driven** approach with separate functions for each conversion type such as length, mass, temperature, time, and volume.

Each module was coded, compiled, and tested to ensure correct conversions. Errors were debugged and the logic optimized for smooth execution. Finally, the program was validated with real examples to confirm **accuracy and reliability.**

4.4. Testing and Validation

The Program was thoroughly tested to ensure accuracy and reliability. Each conversion module was first unit tested individually, followed by integration testing of the complete program. All outputs were compared with manual calculations to validate correctness. The program performed successfully without logical or runtime errors..

4.5. Result and Documentation

The Program was successfully executed, providing accurate and quick conversions across different units. The results matched expected values, proving the program's correctness. All steps, algorithms, and outputs were properly documented for easy understanding and future reference.

4.6. Algorithm

An **algorithm** is a **step-by-step logical procedure** used to solve a specific problem or perform a computation.In programming, it defines the exact sequence of actions the computer must follow to achieve the desired output from a given set of inputs.

For the "**Unite convertor suite"** the algorithm plays a key role in automating the billing process by defining how the program accepts the value and provide its desired output according to the user choice

Purpose of the Algorithm

The purpose of using an algorithm in this project is to:

- The purpose of the algorithm is to provide a clear and step-by-step method for converting one unit of measurement to another. It helps in organizing the program's logic, ensuring accurate calculations, easy debugging, and smooth execution of the Unit Converter Suite Program

Algorithm in the Project:

1. Start the program

2. Display the main menu with unit categories (Length, Mass, Temperature, Time, Volume).

3. Read the user's choice.

4. Ask for the value and unit to convert from

5. Apply the corresponding conversion formula.

6. Display the converted result.

7. Ask if the user wants another conversion.

8. Repeat steps 2–7 if yes, otherwise exit the program.

9. Stop the process.

Importance of Algorithm in the Project:

1.Provides a clear step-by-step solution.

2.Ensures accuracy and consistency in results.

3.Makes the program easier to understand and develop.

4.Helps in detecting and fixing errors quickly.

5.Improves the efficiency and speed of execution.

6.Simplifies complex problems into smaller tasks.

7.Supports easy modification and maintenance of the code.

4.7. Flowchart

A **flowchart** is a **graphicalrepresentation** of a process that shows the sequence of steps using symbols and arrows to indicate the flow of control.

Explanation of Flowcchart

1. Start – Program begins**.**

2. DisplayMenu– Shows unit types (length, weight, temperature).

3. Input Choice– User selects a conversion type.

4. Input Value–User enters the number to convert.

5. ProcessConversion– Program applies the correct formula.

6. Display Result – Shows converted value**.**

7. Repeat or Exit – User chooses to convert again or end the program.

8. End – Program stops**.**

1. SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>

void lengthConverter();
void temperatureConverter();
void weightConverter();
void timeConverter();

int main() {
    int choice;
    do {
        printf("\n==============================\n");
        printf("     UNIT CONVERTER SUITE     \n");
        printf("==============================\n");
        printf("1. Length Converter\n");
        printf("2. Temperature Converter\n");
        printf("3. Weight Converter\n");
        printf("4. Time Converter\n");
        printf("5. Exit\n");
        printf("------------------------------\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: lengthConverter(); break;
```

```
            case 2: temperatureConverter(); break;
            case 3: weightConverter(); break;
            case 4: timeConverter(); break;
            case 5: printf("\nExiting program... Thank you!\n"); break;
            default: printf("\nInvalid choice! Please try again.\n");
        }
    }while (choice != 5);

    return 0;
}

//-------------------------------------------
// LENGTH CONVERTER
//-------------------------------------------
void lengthConverter() {
    int ch;
    float meter, kilometer, centimeter;

    printf("\n--- Length Converter ---\n");
    printf("1. Meter to Kilometer\n");
    printf("2. Kilometer to Meter\n");
    printf("3. Centimeter to Meter\n");
    printf("4. Meter to Centimeter\n");
    printf("Enter your choice: ");
    scanf("%d", &ch);

    switch (ch) {
        case 1:
            printf("Enter length in meters: ");
            scanf("%f", &meter);
```

```
            printf("Length in kilometers: %.3f km\n", meter / 1000);
            break;
        case 2:
            printf("Enter length in kilometers: ");
            scanf("%f", &kilometer);
            printf("Length in meters: %.3f m\n", kilometer * 1000);
            break;
        case 3:
            printf("Enter length in centimeters: ");
            scanf("%f", &centimeter);
            printf("Length in meters: %.3f m\n", centimeter / 100);
            break;
        case 4:
            printf("Enter length in meters: ");
            scanf("%f", &meter);
            printf("Length in centimeters: %.3f cm\n", meter * 100);
            break;
        default:
            printf("Invalid choice!\n");
    }
}

//-----------------------------------------
// TEMPERATURE CONVERTER
//-----------------------------------------
void temperatureConverter() {
    int ch;
    float celsius, fahrenheit;

    printf("\n--- Temperature Converter ---\n");
```

```
    printf("1. Celsius to Fahrenheit\n");
    printf("2. Fahrenheit to Celsius\n");
    printf("Enter your choice: ");
    scanf("%d", &ch);

    switch (ch) {
        case 1:
            printf("Enter temperature in Celsius: ");
            scanf("%f", &celsius);
            fahrenheit = (celsius * 9 / 5) + 32;
            printf("Temperature in Fahrenheit: %.2f°F\n", fahrenheit);
            break;
        case 2:
            printf("Enter temperature in Fahrenheit: ");
            scanf("%f", &fahrenheit);
            celsius = (fahrenheit - 32) * 5 / 9;
            printf("Temperature in Celsius: %.2f°C\n", celsius);
            break;
        default:
            printf("Invalid choice!\n");
    }
}

//-----------------------------------------
// WEIGHT CONVERTER
//-----------------------------------------
void weightConverter() {
    int ch;
    float kg, gram, pound;
```

```
printf("\n--- Weight Converter ---\n");
printf("1. Kilogram to Gram\n");
printf("2. Gram to Kilogram\n");
printf("3. Kilogram to Pound\n");
printf("4. Pound to Kilogram\n");
printf("Enter your choice: ");
scanf("%d", &ch);

 switch (ch) {
    case 1:
        printf("Enter weight in kilograms: ");
        scanf("%f", &kg);
        printf("Weight in grams: %.3f g\n", kg * 1000);
        break;
    case 2:
        printf("Enter weight in grams: ");
        scanf("%f", &gram);
        printf("Weight in kilograms: %.3f kg\n", gram / 1000);
        break;
    case 3:
        printf("Enter weight in kilograms: ");
        scanf("%f", &kg);
        printf("Weight in pounds: %.3f lbs\n", kg * 2.20462);
        break;
    case 4:
        printf("Enter weight in pounds: ");
        scanf("%f", &pound);
        printf("Weight in kilograms: %.3f kg\n", pound / 2.20462);
        break;
    default:
```

```
            printf("Invalid choice!\n");
    }
}

//-----------------------------------------
// TIME CONVERTER
//-----------------------------------------
void timeConverter() {
    int ch;
    float hours, minutes, seconds;

    printf("\n--- Time Converter ---\n");
    printf("1. Hours to Minutes\n");
    printf("2. Minutes to Hours\n");
    printf("3. Minutes to Seconds\n");
    printf("4. Seconds to Minutes\n");
    printf("Enter your choice: ");
    scanf("%d", &ch);

    switch (ch) {
        case 1:
            printf("Enter time in hours: ");
            scanf("%f", &hours);
            printf("Time in minutes: %.2f min\n", hours * 60);
            break;
        case 2:
            printf("Enter time in minutes: ");
            scanf("%f", &minutes);
            printf("Time in hours: %.2f hr\n", minutes / 60);
            break;
```

```
        case 3:
            printf("Enter time in minutes: ");
            scanf("%f", &minutes);
            printf("Time in seconds: %.2f sec\n", minutes * 60);
            break;
        case 4:
            printf("Enter time in seconds: ");
            scanf("%f", &seconds);
            printf("Time in minutes: %.2f min\n", seconds / 60);
            break;
        default:
            printf("Invalid choice!\n");
    }
}
```

SOURCE CODE EXPLANATION:

1. Header Files

```
#include <stdio.h>
#include <stdlib.h>
```

These libraries are used for:

stdio.h → for input/output functions like printf() and scanf().

stdlib.h → for general utility functions (optional here, but good to include).

◆ **2. Function Declarations**

```
void lengthConverter();
void temperatureConverter();
```

```
void weightConverter();
void timeConverter();
```

These are function prototypes for each converter module.

Each function handles one type of conversion.

◆ 3. Main Function

```
int main() {
    int choice;
    do {
        // menu display
        // user choice input
        // switch-case to call selected converter
    } while (choice != 5);
}
```

Displays the main menu repeatedly using a do-while loop.

Accepts user input for conversion type.

Calls the appropriate converter function using a switch-case.

Ends when the user selects 5 (Exit).

◆ 4. Length Converter Function

```
void lengthConverter() {
// Choices: meter ↔ kilometer ↔ centimeter
}
```

Handles length conversions.

Uses formulas like:

Meter → Kilometer: meter / 1000

Kilometer → Meter: kilometer * 1000

Centimeter → Meter: centimeter / 100

Meter → Centimeter: meter * 100

🔹 5. Temperature Converter Function

```
void temperatureConverter() {
    // Choices: Celsius ↔ Fahrenheit
}
```

Handles temperature conversions using formulas:

Celsius → Fahrenheit: (celsius * 9/5) + 32

Fahrenheit → Celsius: (fahrenheit - 32) * 5/9

🔹 6. Weight Converter Function

```
void weightConverter() {
// Choices: kg ↔ g ↔ pounds
}
```

Handles weight conversions like:

Kilogram → Gram: kg * 1000

Gram → Kilogram: gram / 1000

Kilogram → Pound: kg * 2.20462

Pound → Kilogram: pound / 2.20462

🔹 7. Time Converter Function

```
void timeConverter() {
// Choices: hours ↔ minutes ↔ seconds
}
```

Handles time conversions like:

Hours → Minutes: hours * 60

Minutes → Hours: minutes / 60

Minutes → Seconds: minutes * 60

Seconds → Minutes: seconds / 60

🔹 8. Exit Option

When the user selects 5, the program prints:

Exiting program... Thank you!

and ends gracefully.

Control Flow Summary:

1. The program starts and displays a main menu.
2. User selects a conversion type → the program calls the respective function.
3. Inside each function, user enters values → the program performs the conversion and displays the result.
4. After completion, it asks for another conversion.
5. If user chooses Exit, the program terminates.

1. INPUT AND OUTPUT SCREENS

Input Screen 1:

```
 ========================
   UNIT CONVERTER SUITE
 ========================
1. Length Converter
2. Temperature Converter
3. Weight Converter
4. Time Converter
5. Exit
------------------------
Enter your choice: 1

--- Length Converter ---
1. Meter to Kilometer
2. Kilometer to Meter
3. Centimeter to Meter
4. Meter to Centimeter

Enter your choice: 1
Enter length in meters: 500
Length in kilometers:
0.500 km
```

Input screen 2:

```
========================
  UNIT CONVERTER SUITE
========================
1. Length Converter
2. Temperature Converter
3. Weight Converter
4. Time Converter
5. Exit
------------------------
Enter your choice: 2

Enter the temperature to
convert:
98.6

Select the conversion type:
1. Celsius to Fahrenheit
2. Fahrenheit to Celsius

The result is 37.0 °C
```

Output Screen:

```
========================
  UNIT CONVERTER SUITE
========================
1. Length Converter
2. Temperature Converter
3. Weight Converter
4. Time Converter
5. Exit
------------------------
Enter your choice: 5



Exiting program...
```

DISCUSSION & CONCLUSION

7.1 Discussion

The Unit Converter Suite program works well for converting different units like length, temperature, weight, and time. It is easy to use and gives accurate results for all conversions. The use of functions and switch statements makes the program **simple and clear.** Overall, the program helps users understand and convert values easily between different units.

7.2 Conclusion

The project **successfully achieves its goal of simplifying unit conversions.** Ithelpsusersquicklyconvert betweenvariousmeasurementsystemswith **minimal effort**.Theprogramisefficient, **easy to understand**,and can befurtherexpandedtoincludeadditional conversioncategoriesinthefuture.

1. REFERENCES

1. Let Us C by Yashavant P. Kanetkar – BPB Publications
2. Programming in ANSI C by E. Balagurusamy – McGraw Hill Education
3. TutorialsPoint – www.tutorialspoint.com
4. GeeksforGeeks – www.geeksforgeeks.org
5. C Programming Documentation – https://en.cppreference.com